
DATABASE SCHEMA & ARCHITECTURE

Garment Production Tracker

Version 2.0 — Supabase / PostgreSQL

This document defines the complete Supabase PostgreSQL schema, Row Level Security policies, and Edge Function responsibilities for the Garment Production Tracker MVP.

1. Tables Overview

Table	Purpose	Primary writers
<code>purchase_orders</code>	Master PO record and status	Planner (create), Accounts (close)
<code>bom_items</code>	Bill of Materials per PO	Edge Function (auto-calculated)
<code>inventory</code>	Current fabric/trim stock	Planner (setup), Store (deduct)
<code>production_stages</code>	Timestamped floor stage log	Floor staff (mobile)
<code>qc_inspections</code>	QC Pass/Fail records	Floor supervisor
<code>payments</code>	Payment ledger entries	Accounts Manager
<code>delivery_proofs</code>	POD image storage refs	Delivery coordinator
<code>users (auth.users)</code>	Managed by Supabase Auth	Super Admin

2. Table Definitions

2.1 `purchase_orders`

Column	Type	Notes
<code>id</code>	<code>uuid PK</code>	<code>gen_random_uuid()</code> default
<code>po_number</code>	<code>text UNIQUE</code>	Human-readable e.g. PO-2024-001
<code>customer_name</code>	<code>text NOT NULL</code>	
<code>sku_list</code>	<code>jsonb</code>	Array of {sku, size, qty} objects
<code>delivery_date</code>	<code>date</code>	Target delivery
<code>status</code>	<code>text</code>	Enum: draft, in_production, pending_stock, qc, packing, dispatched, closed
<code>courier_name</code>	<code>text</code>	Filled at dispatch
<code>tracking_number</code>	<code>text</code>	Filled at dispatch
<code>created_by</code>	<code>uuid FK → auth.users</code>	Planner who created the PO
<code>created_at</code>	<code>timestampz</code>	<code>now()</code> default
<code>updated_at</code>	<code>timestampz</code>	Updated via trigger

2.2 `bom_items`

Column	Type	Notes
<code>id</code>	<code>uuid PK</code>	

Column	Type	Notes
<code>po_id</code>	<code>uuid FK → purchase_orders</code>	CASCADE on delete
<code>material_name</code>	<code>text NOT NULL</code>	e.g. Cotton Fabric
<code>unit</code>	<code>text</code>	e.g. meters, pieces
<code>required_qty</code>	<code>numeric NOT NULL</code>	Calculated by Edge Function
<code>created_at</code>	<code>timestampz</code>	<code>now()</code> default

2.3 inventory

Column	Type	Notes
<code>id</code>	<code>uuid PK</code>	
<code>material_name</code>	<code>text NOT NULL UNIQUE</code>	
<code>unit</code>	<code>text NOT NULL</code>	
<code>quantity_on_hand</code>	<code>numeric NOT NULL DEFAULT 0</code>	
<code>low_stock_threshold</code>	<code>numeric</code>	UI shows amber warning below this
<code>updated_at</code>	<code>timestampz</code>	

2.4 production_stages

Column	Type	Notes
<code>id</code>	<code>uuid PK</code>	
<code>po_id</code>	<code>uuid FK → purchase_orders</code>	
<code>stage_name</code>	<code>text NOT NULL</code>	<code>material_released</code> , <code>cutting_complete</code> , <code>stitching_complete</code> , <code>kaj_buttoning_complete</code> , <code>ironing_complete</code> , <code>packing_ready</code>
<code>completed_by</code>	<code>uuid FK → auth.users</code>	Floor staff who tapped
<code>completed_at</code>	<code>timestampz</code>	Client timestamp (offline-safe)
<code>synced_at</code>	<code>timestampz</code>	Server receipt time

2.5 qc_inspections

Column	Type	Notes
<code>id</code>	<code>uuid PK</code>	
<code>po_id</code>	<code>uuid FK → purchase_orders</code>	
<code>result</code>	<code>text NOT NULL</code>	pass or fail
<code>rework_note</code>	<code>text</code>	Required if result = fail
<code>inspected_by</code>	<code>uuid FK → auth.users</code>	
<code>inspected_at</code>	<code>timestampz</code>	

2.6 payments

Column	Type	Notes
<code>id</code>	<code>uuid PK</code>	
<code>po_id</code>	<code>uuid FK → purchase_orders</code>	
<code>amount</code>	<code>numeric NOT NULL</code>	
<code>payment_date</code>	<code>date NOT NULL</code>	
<code>note</code>	<code>text</code>	Optional reference / cheque number
<code>logged_by</code>	<code>uuid FK → auth.users</code>	
<code>created_at</code>	<code>timestampz</code>	

2.7 delivery_proofs

Column	Type	Notes
<code>id</code>	<code>uuid PK</code>	
<code>po_id</code>	<code>uuid FK → purchase_orders</code>	
<code>storage_path</code>	<code>text NOT NULL</code>	Supabase Storage object path
<code>public_url</code>	<code>text</code>	Resolved URL for display
<code>uploaded_by</code>	<code>uuid FK → auth.users</code>	
<code>uploaded_at</code>	<code>timestampz</code>	

3. Row Level Security — Key Policies

Enable RLS on every table. The role is read from the user's JWT: `auth.jwt()->'role'`

```
-- Example: purchase_orders table
ALTER TABLE purchase_orders ENABLE ROW LEVEL SECURITY;

-- Directors and above: read all
CREATE POLICY "read_all_orders"
  ON purchase_orders FOR SELECT
  USING (auth.jwt()->'role' IN ('super_admin', 'director', 'planner',
    'accounts'));

-- Floor staff: read in_production orders only
CREATE POLICY "floor_read_active"
  ON purchase_orders FOR SELECT
  USING (auth.jwt()->'role' = 'floor' AND status = 'in_production');

-- Planners: insert new orders
CREATE POLICY "planner_insert"
  ON purchase_orders FOR INSERT
  WITH CHECK (auth.jwt()->'role' = 'planner');

-- Accounts: update status to closed
CREATE POLICY "accounts_close"
  ON purchase_orders FOR UPDATE
  USING (auth.jwt()->'role' = 'accounts');
```

4. Edge Functions

Function	Responsibility
calculate_bom	Called after PO creation. Reads sku_list, multiplies by material ratios from a config table, inserts rows into bom_items, and checks inventory.
generate_pdf	Accepts a po_id, builds a PDF Delivery Challan using Deno + pdftmake, saves to Supabase Storage, returns the public URL.
check_inventory	Called by calculate_bom. Compares bom_items totals against inventory. Returns status 'ok' or 'short' with a list of short materials.

5. Offline Sync Strategy (Floor Mobile)

Floor supervisors may lose WiFi. The following client-side strategy handles this without Firebase's offline persistence:

1. When a supervisor taps a stage button, write the event to `localStorage` immediately (key: `pending_stages`, value: JSON array).
2. Attempt the Supabase upsert in the background. If it succeeds, remove the entry from `localStorage`.
3. Add a window event listener for 'online'. When it fires, flush all `pending_stages` entries to Supabase in order.
4. Show a small badge in the UI (e.g. '2 pending') when `localStorage` has unsynced entries so the supervisor knows the state.



Use `completed_at` (client timestamp) and `synced_at` (server receipt) as separate columns so the stage timeline remains accurate even after a delayed sync.

6. Supabase Storage Buckets

Bucket name	Access	Contents
<code>delivery_proofs</code>	Private (signed URLs)	POD photos uploaded from mobile
<code>challans</code>	Private (signed URLs)	Generated PDF Delivery Challans